

Rushad Wankadia

| 602-802-4929 | wrushad@gmail.com | linkedin.com/in/rushad-wankadia | github.com/RushadW |
Open to Relocate

EDUCATION

Arizona State University – Master's in Software Engineering – GPA: 3.97/4.0 August 2024 – May 2026
SVKM's NMIMS MPSTME University, India – Bachelor's in Computer Engineering – GPA: 3.18/4.0 July 2014 – June 2020

TECHNICAL SKILLS

Languages: Python, C, C++, C#, Java, SQL, Prolog, HTML5, CSS3, JavaScript
Libraries: Numpy, Pandas, scikit-learn, Tensorflow, PyTorch, SQLite, Tkinter
Frameworks: .NET Framework, Flask, FastAPI, React, Vue.js, SpringBoot, GraphQL, REST API, Django
Technologies & Tools: Docker, Git, JWT, Linux, PostgreSQL, Dynatrace, AWS (EC2, SQS, S3, ECR, Lambda), IoT Greengrass

EXPERIENCE

Software Engineer Intern | Rocket Mortgage May 2026 – Present
• Optimizing and extending an existing data-governance backend, refactoring REST APIs, and resolving data-validation bugs to reliably enforce complex legal-hold overrides during GDPR/CCPA deletion requests.

Software Engineer Intern | Rocket Mortgage May 2025 – August 2025
• Built a Dynatrace dashboard from scratch, improving application observability and incident visibility for 9 engineers and leading migration to Metrics Grail.
• Updated Terraform modules to reduce the cloud attack surface, strengthening infrastructure security and compliance.

Junior Software Engineer | Electronics and Engineering Company of India June 2020 – May 2022
• Engineered real-time sensor data visualization for resource-constrained handheld devices in C# and C++, reducing latency from 100-200ms to 15ms via double buffering and cached-layer rendering.
• Designed a hardware abstraction layer decoupling business logic from device constraints, reducing new-variant integration to as little as 4 hours including testing across 4 device configurations.
• Standardized Git workflows for a team of 11, authoring documentation that accelerated onboarding and mentorship.

Software Engineer Intern | Electronics and Engineering Company of India July 2019 – May 2020
• Architected a shared C# library for user-facing and factory applications handling firmware flashing and address-space initialization, separating device configuration from runtime logic and reducing new-device onboarding to a single half-day.
• Eliminated approximately 900 redundant polling calls per minute across 30 machines by migrating to a pub-sub model, capturing data only on state change.

PROJECTS

RxRefactor – AI Medical Benefit Drug Policy Tracker | *React, Python, FastAPI, ChromaDB, Anthropic API, Docker*
• Delivered an AI-powered medical benefit drug policy tracker covering 55 policies across 5 major insurers, consisting of thousands of pages, accounting for over 6,100 searchable semantic policy fragments.
• Architected a hybrid retrieval system combining structured exact-match lookups with RAG-based semantic search, returning policy interpretations in under 500ms end-to-end.
• Implemented policy evolution tracking with version history and field-level diffs (backend API + UI) to audit and compare policy changes over time.

Assessment AI – LLM-Powered Student Assessment Platform | *React, Python, Django, SQL, OpenAI API, Gemini API, Docker*
• Engineered a Django REST Framework backend with JWT authentication, custom user profiles, dual-LLM pipeline (GPT-4.1 Mini primary, Gemini 2.5 Flash fallback), and MySQL storage, deployed as a microservice on GCP Cloud Run.
• Designed a two-tier retrieval layer using O(1) SHA-256 exact-match and character n-gram vector similarity scoring to rank and serve cached assessments for semantically equivalent student answers, reducing redundant LLM calls.
• Implemented per-student conversation memory using Django ORM-retrieved assessment history injected as structured context into OpenAI/Gemini tutor prompts, enabling targeted follow-up Q&A and guided remediation.
• Authored comprehensive automated tests and validation suites for assessment caching, LLM fallback routing, and conversation memory to ensure reliability and correctness.

Face Recognition Pipeline – IaaS, Serverless & Edge Architectures | *PyTorch, FaceNet, Lambda, IoT Greengrass, Docker, Python*
• Engineered a PyTorch inference pipeline (pretrained FaceNet, MTCNN detection, min-max normalization) with a custom autoscaler (0–15 EC2 instances, scales to 0 in 0.24s) and dual SQS queues, achieving 0.96s avg latency across 100 concurrent requests.
• Re-architected as a serverless pipeline using SQS-triggered Lambda functions containerized via Docker/ECR, achieving 100% prediction accuracy under concurrent load.
• Extended to an edge/cloud split via IoT Greengrass and MQTT, where edge inference reduced latency 57% over the serverless variant (from 1.78s to 0.77s) and short-circuited cloud Lambda invocations for no-face frames.